



2E10 Final Report (Part C)

Group X15

Introduction

This report outlines the communication framework between a Supervisor PC and an Arduino Buggy, including messages, signals, events and feedback between the components and the buggy hardware interface. Our system involves sending commands from the Supervisor PC to the Arduino Buggy, for which the buggy will be controlled by the supervisor PC by simply pressing buttons, and while the buggy is in action, it will send information back to the PC. This ensures smooth operation and responsiveness of the buggy.

System Overview

The communication between the Supervisor PC and the Arduino Buggy is facilitated through a wireless interface (via Wi-Fi). The Supervisor PC acts as a controller, issuing commands, while the Arduino Buggy responds by executing actions and returning system status.

Components Involved:

- Supervisor PC: Sends commands and processes feedback.
- Arduino Buggy: Executes commands and returns status updates.
- Sensors: Detect obstacles and provide real-time feedback.
- Motors: Control the movement of the buggy.
- Encoders: Used to count the number of wheel pulses.

Communication Protocol

Messages Sent by the Supervisor PC

- START - Begin buggy movement forward.
- STOP – Stop all movement immediately.
- MODE - Pick buggy operating mode (Object Following/Reference Speed)
- REF SPEED - Sends buggy target speed in Reference Speed mode.

Messages Sent by the Arduino Buggy back to the PC.

- Motion status - Sends the motion of the buggy, moving forward, left or right.
- Obstacle detecting - Stops automatically when an obstacle is detected in the path.
- Distance travelled - Sends back the distance travelled.
- Current Speed - Sends back the current operating speed of the buggy.

Control Flow Algorithm (Pseudo Code):

Include WiFiS3 Library

Set WiFi SSID and Password Constants

Create Server object on port 5200

Define Global Constants:

- Wheel Circumference
- Distance covered every encoder tick
- Left and Right IR Sensor and US Sensor Pins
- Minimum and Max PWM values

Define Global Variables:

- IR sensor states (Left/Right)
- Buggy direction state (e.g., Forward, Left, Right, Stopped)
- Movement flags for efficiency (e.g. is buggy already moving)
- Buggy Operating Mode

Create Encoder Class:

- Initialize encoder position and covered distance to 0
- Declare encoder pin
- Define method to calculate distance from pulses using wheel circumference

Create Ultrasonic Sensor Class:

- Declare Trigger and Echo pins
- Define variables for distance measurement and duration
- Create method to measure object distance using ultrasonic pulses

Create Motor Class:

- Declare left, right, and speed (PWM) pins
- Define methods for:
 - Moving Forward
 - Reversing
 - Stopping Motor
 - Changing Speed (Constrained between 0 and Max PWM)

Create a function to check if received data from buggy is numeric or not

Create Objects:

- Left Motor
- Right Motor
- Ultrasonic Sensor
- Left Encoder
- Right Encoder

Initialize Timer Variable to 0

Declaring and Initializing related PID constants and variables (e.g. Kp, Ki, Previous Errors, etc..)

Defining PID control function to correct output based on current error between target speed and current speed, accumulated previous errors and error rate of change.

Define function to give a PWM boost to motors incase it's been idle for too long based on time between encoder pulses when it shouldn't be (i.e. when it's not stopped)

Define Interrupt Service Routines (ISR):

- Left encoder ISR: Increment left encoder pulse count
- Right encoder ISR: Increment right encoder pulse count
- Both encoders calculate time intervals between pulses, noise filtering very small values (< 0.02 s)

Setup:

- Begin Serial communication at 9600 baud
- Start WiFi communication and attempt connection until successful
- Print WiFi connection status and IP Address
- Configure motor and sensor pins with appropriate modes (INPUT/OUTPUT)

- Attach interrupt functions to encoder pins
- Initialise variables such as received buggy data and speed values to 0

Loop:

- Create WiFi Client and loop until connected
- While connected: (if disconnected stop motors immediately until reconnected)
 - Check received data from server (PC)
 - If Go command is received:
 - Check for obstacles every 0.25s using US sensor, millis() and timer
 - Check for chosen mode (Object Following/Reference Speed)
 - If obstacle detected less than 10 cm stop and report its distance
 - Else if object following map target speed between 10 and 20 cm/s based on object distance from 10 to 30 cm (e.g. if object at 20cm, set target speed to 15 cm/s, if at 10 set to 10 cm/s)
 - Else if reference speed mode set target speed to reference speed
 - Every 100 ms call PID function to adjust speed based on target speed and give motors PWM boost if idle for > 500 ms
 - If both sensors on black, go forward
 - If right sensor on black, go left
 - If left sensor on black, go right
 - Set all relevant flags and report status in all cases and send speed value
- If Stop command is received:
 - Stop both motors and report the travelled distance to GUI.

Conclusion:

This report provides a structured approach to establishing communication between the Supervisor PC and the Arduino Buggy in detail. By outlining the message, signals and data structures involved, it gives us a better understanding of the system between the buggy and PC's operation. It seamlessly switches between object following and reference speed modes and is overall very responsive to commands issued by our PC.

The system could be further improved upon by fine tuning values (i.e. PID constants, Kp, Ki and Kd as well as PWM values) such that movement is a bit smoother but it otherwise functions properly, slowing/stopping when in the vicinity of an object and changing speeds according to the given reference speed. Furthermore, readings such as distance and speed are very accurate and effective PID control was achieved when moving on straight lines. Finally, the buggy could have been optimised by further experimentation to find the most responsive settings at least given the context of our application.